

WATERMARK FORGERY: FORGING IMAGE WATERMARKS

Mark Stephen Thomas
Universität des Saarlandes
Saarbrücken, Germany
Matriculation No: 7081771

Yogesh Kishor Kulkarni
Universität des Saarlandes
Saarbrücken, Germany
Matriculation No: 7083560

1 INTRODUCTION

Image watermarking embeds an imperceptible, detectable signal into an image to certify its provenance. We are given, for each of eight undisclosed watermarking methods $WM_1, \dots, WM_8$, a set of 25 source images carrying the same hidden message and 200 clean target images. The goal is to forge: transfer each method’s watermark onto a fixed block of 25 targets so the forged images are detected as watermarked while remaining visually faithful. Each forged image is scored as $S = S_{\text{det}} \times S_{\text{qit}}$, where $S_{\text{det}} = \max(0, 2(\text{BitAccuracy} - 0.5))$ and $S_{\text{qit}} = \exp(-8 \cdot \text{LPIPS})$; the final score averages over all 200 images. Because the score is a product, a batch must win both detection and quality: zero detection yields zero regardless of visual fidelity.

2 APPROACH

Central observation: The detector grades the decoded message bits (BitAccuracy), not mere watermark presence. Averaging the source residual onto targets reproduces watermark presence but not the specific message, capping the detection score near zero. The only way to earn S_{det} is to reproduce each method’s actual message, so we treat each batch according to its scheme type.

Classical schemes: identify, recover, re-embed: Probing each batch with the invisible-watermark library, we identified WM_1 as **dwtDct** (16-bit) and WM_2 as **RivaGAN** (32-bit). For each, we decode all 25 sources, take the per-bit majority vote to recover the message, and re-embed into the clean targets with the genuine encoder. This achieves decode-consistency of 0.91 (WM_1) and 0.99 (WM_2).

Diffusion pixel-leak: fixed-pattern transplant: WM_5 is a SERUM-style (1) diffusion watermark whose fixed per-user pattern W partially leaks into pixel space. We estimate W as the mean edge-preserving high-frequency residual across sources, $W = \frac{1}{25} \sum_i (s_i - \text{bilateral}(s_i))$, and transplant it additively: $t' = \text{clip}(t + \beta W)$. A strength of $\beta = 2.0$ maximised the leaderboard contribution.

Quality optimisation: Since re-embedded watermarks retain large detection margin (BitAccuracy ≈ 0.9 – 1.0), we blend each watermarked image back toward clean: $f = \alpha w + (1 - \alpha)c$. Using offline LPIPS measurement, we found the $S_{\text{det}} \times S_{\text{qit}}$ product peaks at $\alpha = 0.85$ for WM_1 and $\alpha = 0.50$ for WM_2 (RivaGAN tolerates aggressive blending while keeping BitAccuracy ≈ 0.99).

Remaining batches: $WM_3, WM_4, WM_6, WM_7, WM_8$ do not respond to any pixel-space forging (Section 4); they receive a low-strength averaged residual ($\alpha=0.4$) which contributes negligible detection but preserves quality better than stronger blending.

3 RESULTS

Table 1 tracks leaderboard score across builds.

Key findings: (1) Watermark *presence* is easy to copy but the *message* is not; re-embedding with the genuine encoder is required to move BitAccuracy above chance.

(2) Exactly one deep-learning batch (WM_5) leaks a fixed, transplantable pattern into pixel space.

(3) Because scoring is a product, optimising quality on detected batches is as valuable as improving detection.

Build	Description	Score
forge_classical	WM ₁ /WM ₂ re-embed; WM ₃₋₈ avg $\alpha=0.5$	0.423
forge_classical_serum	+ WM ₅ SERUM transplant $\beta=1.0$; avg $\alpha=0.4$	0.457
forge_final	+ quality-blend WM ₁ (0.85)/WM ₂ (0.50); $\beta=2.0$	0.463

Table 1: Leaderboard progression across committed builds. Each row adds one improvement: classical re-embedding, SERUM pixel-leak transplant, and product-aware quality optimisation.

4 WHAT WE RULED OUT

Forging WM₃₋₄ and WM₆₋₈ requires reproducing a message absent from pixel space. Using a clean-target control for every probe, we systematically tested classical decoders (dwtDct, dwtDctSvd, RivaGAN at various lengths), TrustMark (Q/P/B variants), and the blind-watermark library, none yielded a consistent message across the 25 sources beyond random agreement. We additionally ran the published detectors for two candidate families end-to-end: BitMark (2) (official code, Infinity tokeniser) returned z -scores near zero on all batches, and the QuantLoss provenance signal of (3) (LlamaGen tokeniser) showed no gap between sources and clean targets. Further pixel-space and frequency-domain analyses (block-DCT consistency, Fourier-phase tests, LSB analysis) were equally flat. These consistent negatives indicate the remaining watermarks live in a generative model’s latent or token space and require model-specific components to forge.

5 CONCLUSION

Our best submission attains a leaderboard score of **0.463** by cracking three of the eight schemes: re-embedding WM₁ (dwtDct) and WM₂ (RivaGAN) with their genuine encoders, and transplanting the pixel-space leak of WM₅ (SERUM-style), with product-aware quality optimisation. The five remaining schemes carry no pixel-space signal and are latent or token-domain watermarks whose detectors are model-specific networks.

Practical implications: The ease with which classical watermarks were forged, requiring only a public library and 25 example images, raises serious concerns for real-world provenance systems. Any content-authentication pipeline built on signal-processing watermarks (DWT, DCT, or GAN-based encoders) is vulnerable to forgery by anyone with sample access, undermining trust in watermarked media. Conversely, latent-space watermarks proved substantially harder to forge, suggesting that embedding watermarks inside a generative model’s representation space is a more robust design. However, our SERUM result shows that even model-embedded watermarks can leak detectable patterns into pixel space, so robustness should not be assumed without empirical verification. As watermarking becomes central to AI-generated content regulation, these vulnerabilities demand that deployed systems are rigorously audited against forgery attacks before being relied upon for provenance or copyright enforcement.

Code: https://github.com/MARKST-47/TML-26/tree/main/tml26_task4

REFERENCES

- [1] Kociszewski, T., et al. (2026). SERUM: Simple, Efficient, Robust, and Unifying Marking for Diffusion Models. *ICLR*.
- [2] Zhao, X., et al. (2025). BitMark: Watermarking Bitwise Autoregressive Image Generative Models. *NeurIPS*.
- [3] Zhao, X., Kerner, B., Meintz, L., Bakr, A., Boenisch, F., & Dziedzic, A. (2026). Data Provenance for Image Auto-Regressive Generation. *ICLR*.